

Game Plaza - Risk Breakdown Structure (RBS)

1. TECHNICAL RISKS

1.1 Database Issues

- ****Risk****: Database connection fails or MySQL not running
- ****Impact****: HIGH - Project cannot run
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Provide SQL dump file for easy import
 - Create clear setup instructions
 - Implement error logging for DB connection
 - Action: Test DB connection on startup

1.2 PHP/PDO Compatibility

- ****Risk****: PDO not installed or MySQL driver missing
- ****Impact****: MEDIUM - API endpoints fail
- ****Probability****: LOW
- ****Mitigation****:
 - Use common PDO syntax (compatible with PHP 7.4+)
 - Document required PHP extensions
 - Provide troubleshooting guide
 - Action: Test with PHP 7.4+ minimum

1.3 Session Management Issues

- ****Risk****: Sessions not persisting across pages
- ****Impact****: HIGH - Users get logged out unexpectedly
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Use secure session configuration
 - Implement session timeout handling
 - Add session refresh on API calls
 - Action: Test multi-page navigation

1.4 API Response Errors

- ****Risk****: JSON responses malformed or incomplete
- ****Impact****: MEDIUM - JavaScript fails silently
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Validate all API responses before sending
 - Implement proper error codes (200, 400, 500)
 - Add detailed error messages
 - Action: Test all endpoints with Postman/curl

1.5 JavaScript Fetch Issues

- ****Risk****: CORS, network errors, or API endpoint 404s
- ****Impact****: MEDIUM - Frontend breaks
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Implement try-catch in all fetch calls
 - Add timeout handling

- Show user-friendly error messages
- Action: Test fetch in browser DevTools

2. SECURITY RISKS

2.1 SQL Injection

- ****Risk****: Malicious SQL in user inputs
- ****Impact****: CRITICAL - Database compromise
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Use ONLY prepared statements with PDO
 - Never concatenate user input into queries
 - Validate/sanitize all inputs
 - Action: Code review for SQL queries

2.2 Password Security

- ****Risk****: Passwords stored in plain text or weak hashing
- ****Impact****: CRITICAL - User account compromise
- ****Probability****: LOW (if using password_hash)
- ****Mitigation****:
 - Use password_hash() with bcrypt
 - Use password_verify() for login
 - Never store plain passwords
 - Action: Enforce in auth functions

2.3 Session Hijacking

- ****Risk****: Session ID exposed or stolen
- ****Impact****: CRITICAL - Account takeover
- ****Probability****: LOW (local testing)
- ****Mitigation****:
 - Use secure session flags (HttpOnly, Secure)
 - Regenerate session ID on login
 - Implement CSRF tokens
 - Action: Configure php.ini session settings

2.4 XSS (Cross-Site Scripting)

- ****Risk****: Malicious JavaScript injected in user content
- ****Impact****: MEDIUM - Client-side attacks
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Use htmlspecialchars() for all user output
 - UsetextContent instead of innerHTML in JS
 - Validate on both client and server
 - Action: Audit all output operations

2.5 CSRF (Cross-Site Request Forgery)

- ****Risk****: Unauthorized actions via hidden forms
- ****Impact****: MEDIUM - Unintended user actions
- ****Probability****: LOW (local testing)

- ****Mitigation****:
- Implement CSRF token generation
- Validate CSRF tokens on POST/PUT/DELETE
- Use SameSite cookie attribute
- Action: Add token validation to API

3. DESIGN & LOGIC RISKS

3.1 Matchmaking Algorithm Issues

- ****Risk****: Suggestions don't match shared games properly
- ****Impact****: MEDIUM - Poor user experience
- ****Probability****: MEDIUM
- ****Mitigation****:
- Implement clear matching criteria
- Add logging to debug matches
- Test with sample data
- Action: Unit test matchmaking logic

3.2 Data Model Problems

- ****Risk****: Foreign keys cause integrity issues or slow queries
- ****Impact****: MEDIUM - Data corruption or performance
- ****Probability****: MEDIUM
- ****Mitigation****:
- Use proper foreign key constraints
- Index frequently queried columns
- Test with larger datasets
- Action: Review database schema

3.3 File Upload Risks

- ****Risk****: If profile pics added, malicious uploads
- ****Impact****: MEDIUM - Server compromise
- ****Probability****: LOW (MVP won't have uploads initially)
- ****Mitigation****:
- Validate file types & size
- Store uploads outside web root
- Rename files with random names
- Action: Skip for MVP, add later safely

4. PERFORMANCE RISKS

4.1 N+1 Query Problem

- ****Risk****: Inefficient queries in loops causing slow pages
- ****Impact****: MEDIUM - Slow user experience
- ****Probability****: MEDIUM
- ****Mitigation****:
- Use JOINS instead of loops
- Implement eager loading

- Add database indexes
- Action: Review query patterns

4.2 Large Result Sets

- ****Risk****: Loading 1000+ suggestions causes page hang
- ****Impact****: MEDIUM - Unresponsive UI
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Implement pagination (20-50 results per page)
 - Add limit to query results
 - Use AJAX lazy loading
 - Action: Add LIMIT to all queries

4.3 CSS/JS Load Performance

- ****Risk****: Large CSS/JS files slow page load
- ****Impact****: LOW - Minor UX issue (school project)
- ****Probability****: LOW
- ****Mitigation****:
 - Keep assets minimal
 - Use CDN for Bootstrap/AOS (optional)
 - Minify CSS/JS later if needed
 - Action: Monitor page load times

5. SCOPE & TIMELINE RISKS

5.1 Scope Creep

- ****Risk****: Adding too many features delays MVP
- ****Impact****: HIGH - Project incomplete
- ****Probability****: HIGH
- ****Mitigation****:
 - Stick to MVP features only
 - Document optional features separately
 - Track time spent per task
 - Action: Discipline on feature additions

5.2 Time Underestimation

- ****Risk****: Tasks take longer than planned
- ****Impact****: HIGH - Deadline miss
- ****Probability****: MEDIUM
- ****Mitigation****:
 - Break tasks into smaller chunks
 - Test frequently during development
 - Build in 1-2 buffer days
 - Action: Daily progress tracking

5.3 Integration Issues

- ****Risk****: APIs work in isolation but fail when integrated
- ****Impact****: MEDIUM - Late-stage bugs
- ****Probability****: MEDIUM

- ****Mitigation****:
- Integrate early and often
- Test end-to-end flows frequently
- Use version control (Git)
- Action: Commit working code daily

6. INFRASTRUCTURE RISKS

6.1 Local Development Environment

- ****Risk****: XAMPP/Laragon crashes or files get corrupted
- ****Impact****: MEDIUM - Work loss
- ****Probability****: LOW
- ****Mitigation****:
- Keep backups of SQL dumps
- Use Git for version control
- Test on fresh machine if possible
- Action: Daily Git commits

6.2 Deployment Issues

- ****Risk****: Code works locally but fails on hosting
- ****Impact****: MEDIUM - Grading/Demo problems
- ****Probability****: MEDIUM
- ****Mitigation****:
- Document server requirements
- Test PHP version compatibility
- Create deployment checklist
- Action: Test on actual server before submission

RISK PRIORITY MATRIX

Risk	Severity	Probability	Priority	Owner
SQL Injection	CRITICAL	MEDIUM	P1	Dev
Password Security	CRITICAL	LOW	P1	Dev
Session Hijacking	CRITICAL	LOW	P2	Dev
XSS/CSRF	MEDIUM	MEDIUM	P2	Dev
Scope Creep	HIGH	HIGH	P1	PM
Time Underestimation	HIGH	MEDIUM	P1	PM
DB Connection Failure	HIGH	MEDIUM	P2	DevOps
Matchmaking Logic	MEDIUM	MEDIUM	P3	Dev
Performance/N+1	MEDIUM	MEDIUM	P3	Dev
Integration Issues	MEDIUM	MEDIUM	P3	QA

RISK MONITORING

****Weekly Check-ins:****

- [] Any SQL injection vulnerabilities?
- [] All passwords properly hashed?
- [] Project on schedule?
- [] Any integration issues?
- [] Database backups created?

****Before Deployment:****

- [] Security code review completed
- [] All tests passing
- [] Performance acceptable
- [] Documentation complete
- [] Final backup taken